Currying, Partial Application & Composition

Currying/Composition · Build price pipeline from small functions · Build with Waqa

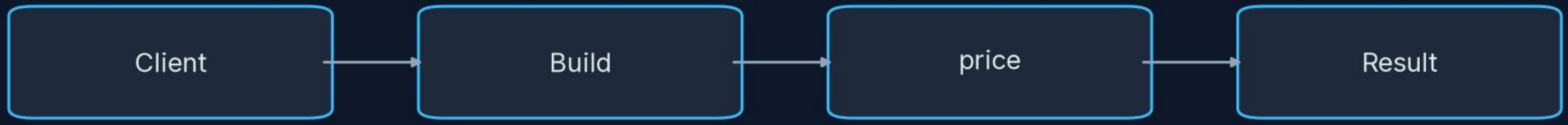
Today's Hook

- Problem: Build price pipeline from small functions
- Pattern: Currying/Composition — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Build price pipeline from small functions
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Currying/Composition: Build price pipeline from small functions

Real-World Scenario

- Running example: Build price pipeline from small functions
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.functions.problem;

import java.util.*;

public final class NaivePrice {
  public static int price(int base, int pct, int flat){ return Math.max(0, base*(100-pct)/100 - flat); }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.functions.problem;

import java.util.*;
import java.util.function.*;

final class Fns {
  static Function<Integer, Function<Integer, Integer>> curryAdd(){ return a -> b -> a+b; }
  static <A,B,C> Function<A,C> compose(Function<B,C> g, Function<A,B> f){ return a -> g.apply(f.apply(a)); }
  static Function<Integer,Integer> pricePipeline(int pct, int flat){
    Function<Integer,Integer> percent = c -> c*(100-pct)/100;
    Function<Integer,Integer> flatOff = c -> Math.max(0, c-flat);
    return compose(flatOff, percent);
  }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.functions.problem;
import java.util.*; import java.util.function.*;
public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Currying/Composition: price pipeline ===");
    System.out.println("--- naive ---"); System.out.println(NaivePrice.price(1000,10,50));
System.out.println("--- curry/compose ---");
System.out.println(Fns.curryAdd().apply(2).apply(3));
System.out.println(Fns.pricePipeline(10,50).apply(1000));
  }
}
```

Walkthrough Output

```
=== Currying/Composition: price pipeline ===
--- naive ---
850
--- curry/compose ---
5
850
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Currying/Composition.

Related Patterns

- Contrast Currying/Composition with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Currying/Composition → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.